From Bellman to Q-Learning

A Complete Derivation: V^{π}, Q^{π}, Optimality Equations, and the Update Rule

Week 5 — Reinforcement Learning: The Idea

From Conway to LangGraph · Dipartimento di Fisica, UniBo

Starting Point: The Two Bellman Equations for Policy π

The two coupled equations (valid for any fixed policy π)

$$V^{\pi}(s) = \Sigma_a \ \pi(a|s) \cdot Q^{\pi}(s, a) \qquad \leftarrow \text{ value = weighted average over actions}$$

$$Q^{\pi}(s, a) = R(s, a) + \gamma \Sigma_{s'} \ P(s'|s,a) \cdot V^{\pi}(s') \qquad \leftarrow \text{ Q = reward + discounted future V}$$

What these equations say

$V^{\pi}(s)$ is the average of Q over all actions, weighted by how often policy π selects each action.

$Q^{\pi}(s, a)$ is the immediate reward plus the discounted value of wherever you land next.

The problem: a free parameter

Both equations contain π as a free parameter. Different policies give different functions. There is an infinite family of solutions — one per policy. The question for optimality: what do these equations become when we choose the best policy?

Define the Optimal Value Functions V* and Q*

Optimise over all possible policies

Among all policies π, define the one(s) that maximise the total discounted reward from every state simultaneously:

$V^*(s) = \max_\pi V^\pi(s)$

Best achievable total reward starting from s

$Q^*(s,a) = \max_\pi Q^\pi(s, a)$

Best reward: force action a, then act optimally

What Q*(s,a) decomposes into

Q*(s,a) has two parts: (1) you take action a in state s — fixed and forced, regardless of what any policy would have chosen; (2) from the resulting state s' onward, you act optimally. The * refers entirely to step (2).

Why Q* is more useful than V*

$V^*(s)$
Tells you how good a state is.
But to extract the best action you still need $P(s'|s,a)$:
$\text{argmax}_a \Sigma P(s'|s,a) V^*(s')$.

$Q^*(s,a)$
Tells you the value of each (s,a) pair directly.
Optimal policy is trivially read off:
$\pi^*(s) = \text{argmax}_a Q^*(s,a)$ — no model needed.

The Crucial Link: $V^*(s) = \max_a Q^*(s,a)$

General policy vs optimal policy: the average collapses to a max

	General policy π	Optimal policy π^*
V in terms of Q	$V^{\pi}(s) = \Sigma_a \pi(a\|s) \cdot Q^{\pi}(s, a)$	**$V^*(s) = \max_a Q^*(s,a)$**
Policy character	Stochastic — mixes actions	Deterministic — one best action
Operation on Q	Weighted average	**Hard maximum**

Why this must be true (not an assumption)

$V^*(s)$ is defined as the maximum of $V^a(s)$ over all π. The optimal policy puts all probability on the single best action: $\pi^*(a|s) = 1$ if $a = \text{argmax } Q^*$, else 0. Inserting this into $\Sigma_a \pi^*(a|s)\, Q^*(s,a)$ immediately gives $\max_a Q^*(s,a)$. No assumptions needed.

Key insight: $V^*(s) = \max_a Q^*(s,a)$ is the bridge that converts the sum-over-π in the general Bellman equation into the max-over-a in the optimality equation. This single substitution is what creates the self-referential fixed-point equation for Q^*.

Derive the Bellman Optimality Equation for Q*

Three substitutions — nothing else needed

1 Start from the general Bellman equation for Q^{π} — valid for any policy, including π^*:

$$Q^{\pi}(s, a) = R(s, a) + \gamma \Sigma_{s'} P(s'|s,a) \cdot V^{\pi}(s')$$

2 Specialise to $\pi = \pi^*$ (the optimal policy):

$$Q^*(s, a) = R(s, a) + \gamma \Sigma_{s'} P(s'|s,a) \cdot V^*(s')$$

3 Substitute $V^*(s') = \max_a Q^*(s',a')$ from the previous slide:

$$\mathbf{Q^*(s, a) = R(s, a) + \gamma \Sigma_{s'} P(s'|s,a) \cdot \max_{a'} Q^*(s', a')}$$

Bellman Optimality Equation for Q — no new assumptions, just three substitutions from the definitions.*

Why 'Tighter'? Q^π vs Q^*

General Q^π

Equation
$Q^\pi = R + \gamma \Sigma P \cdot V^\pi$

Free parameter
Policy π appears on both sides via V^π

Solutions
One Q^π per policy π
∞ many solutions

Physics analogy
Partition function $Z(\beta)$ for a given Hamiltonian

Difficulty

Must specify π first, solve, then compare πs

→

max over π

Optimal Q^*

Equation
$Q^* = R + \gamma \Sigma P \cdot \max_a Q^*$

Free parameter

None — RHS determined by Q^* itself

Solutions
Unique fixed point: $Q^* = T[Q^*]$

Physics analogy
Ground state energy E_0 — unique (non-degenerate)

Difficulty
Self-referential, but has a unique solution

Why Does a Unique Solution Exist? — The Contraction Mapping

Define the Bellman optimality operator T

Define the operator T that maps any Q-function to a new one by applying the RHS of the Bellman equation:

$$(T[Q])(s,a) = R(s,a) + \gamma \Sigma_{s'} P(s' \mid s,a) \cdot \max_{a'} Q(s',a')$$

Finding Q* is equivalent to finding the fixed point T[Q*] = Q*.

The contraction property (γ < 1 is essential)

$$\|T[Q] - T[Q']\|_\infty \leq \gamma \cdot \|Q - Q'\|_\infty \qquad \text{with } \gamma \in [0,1)$$

Each application of T shrinks the sup-norm distance between any two Q-estimates by factor γ. Starting from any Q_0, the sequence $T[Q_0]$, $T^2[Q_0]$, $T^3[Q_0]$, … converges to Q*. This is the Banach fixed-point theorem applied to function space.

Physics analogy — Renormalisation Group: T is an RG flow on the space of Q-functions. The fixed point Q* is the IR fixed point (the universality class). The discount factor γ plays the role of the RG step size — it controls how fast irrelevant operators decay. Q-learning converges by the same mechanism that makes RG fixed points universal: the contraction ensures all trajectories end at the same point, independent of Q_0.

Summary: From V^π / Q^π to Q^* — Complete Comparison

	General policy π	Optimal policy π^*
V in terms of Q	$V^\pi = \Sigma_a \pi(a\|s)\, Q^\pi$	$\mathbf{V^* = \max_a Q^*}$
Q equation	$Q^\pi = R + \gamma\Sigma P(\ldots)\cdot V^\pi$	$\mathbf{Q^* = R + \gamma\Sigma P\cdot \max Q^*}$
Free parameter	Policy π	None
Number of solutions	One per policy (∞ many)	Unique fixed point
Physics analogy	Partition fn at given T	Ground state energy
How to find it	Policy evaluation	Q-learning (next slides)

Part II

The Q-Learning Update Rule

Full Derivation and Justification

C. J. C. H. Watkins (1989) — learning Q from experience without knowing P(s' |s,a)*

Step 1 — What Problem Are We Solving?

The Bellman optimality equation requires R and P — both often unknown

$$Q^*(s,a) = R(s,a) + \gamma \Sigma_{s'} P(s'|s,a) \cdot \max_{a'} Q^*(s',a')$$

Requires R(s,a)

The reward function must be specified for all (s,a). In many real environments this is implicit in the simulator, not a formula.

Requires P(s'|s,a)

The full transition model of the environment. Unavailable for robots, games, most real-world agents.

Q-learning's answer: learn from raw experience

Instead of solving the equation, interact with the environment and collect transitions (s, a, r, s'). The reward r is observed directly. The next state s' is produced by the environment — it samples from P(s'|s,a) implicitly, so we never need P explicitly.

Model-free RL: Replace the intractable expectation ΣP[…] with a single observed transition. Each observed s' is one unbiased sample from P(·|s,a). The noise averages out over many visits to (s,a).

Step 2 — Rewrite the Target as an Expectation

The Bellman equation is already an expectation over s'

The sum $\Sigma_{s'} P(s'|s,a)[\ldots]$ is an expected value over next states. Write it explicitly:

$$Q^*(s,a) = \mathbb{E}_{s'}[\, r + \gamma \max_{a'} Q^*(s',a') \,]$$

Name the quantity inside the expectation: the TD target y

For a single observed transition (s, a, r, s'), the Temporal Difference (TD) target is:

$$y = r + \gamma \cdot \max_{a'} Q(s', a')$$

The Bellman equation now simply says: $Q^*(s,a) = E[y]$. Your $Q(s,a)$ should equal the long-run average of y over all transitions from (s,a).

Physics analogy — Monte Carlo integration: The expectation $E_{s'}[y]$ is an integral over phase space. Estimating it from a single sample s' is exactly a one-sample Monte Carlo estimator. The estimator is unbiased; the variance is large but decreases as $1/\sqrt{N}$ with the number N of independent samples.

Step 3 — The TD Error: Measuring How Wrong You Are

Define δ — the prediction error

If Q = Q*, the Bellman equation says E[y] = Q(s,a). For one transition, define the TD error:

$$\delta = y - Q(s,a) = [\, r + \gamma \max_{a'} Q(s',a') \,] - Q(s,a)$$

δ > 0

Target y is higher than your estimate.
You underestimated (s,a).
⇒ increase Q(s,a).

δ = 0

Target matches estimate.
No error for this sample.
⇒ no update needed.

δ < 0

Target y is lower than estimate.
You overestimated (s,a).
⇒ decrease Q(s,a).

When E[δ] = 0 everywhere, you have found Q*

If Q = Q*, the Bellman equation guarantees E[δ] = 0 for all (s,a). Conversely, if E[δ] = 0 for all (s,a), then Q satisfies the Bellman equation, hence Q = Q*. The TD error is precisely the residual of the fixed-point equation, sampled from one transition.

Step 4 — The Update Rule as Stochastic Gradient Descent

Define a loss function: squared Bellman residual

Measure how far Q is from satisfying the Bellman equation:

$$L(Q) = \mathbb{E}[\,(Q(s,a) - y)^2\,] = \mathbb{E}[\delta^2]$$

Take the gradient with respect to Q(s,a)

Treating the target $y = r + \gamma \max Q(s',a')$ as a fixed constant (the key approximation — see note below), differentiate:

$$\partial L \,/\, \partial Q(s,a) = -2 \cdot \mathbb{E}[\delta]$$

One gradient descent step on a single sample gives the Q-learning rule

$$\mathbf{Q(s,a) \leftarrow Q(s,a) + \alpha\delta = Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]}$$

Note: Treating y as fixed (semi-gradient) makes the update computationally cheap and stable, at the cost of not being true gradient descent — but convergence is still guaranteed.

Step 5 — Anatomy of the Update, Term by Term

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\,r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$$

Term	Symbol	Role
Current estimate	$Q(s,a)$	Your present belief about the value of taking a in state s
Observed reward	r	Hard evidence from the environment — what actually happened this step
Bootstrap estimate	$\gamma \max_{a'} Q(s',a')$	Best current guess about future value — uses your own (imperfect) Q
TD target	$y = r + \gamma \max Q$	Better estimate of $Q^*(s,a)$ than $Q(s,a)$ alone — combines evidence + bootstrap
TD error	$\delta = y - Q(s,a)$	How surprised you were — direction and magnitude of the correction
Learning rate	$\alpha \in (0, 1]$	How much to trust the new evidence vs your current belief

Step 6 — Bootstrapping: Using Your Own Estimate as a Teacher

What 'bootstrapping' means

The TD target uses your own current Q to estimate the future:

$y = r + \gamma \max_{a'} Q(s',a')$ ← Q is imperfect, yet used to improve Q

This is not circular. The contraction property guarantees each iteration brings Q closer to Q* even if Q was wrong to begin with. Bootstrapping from a slightly wrong Q still pulls in the right direction.

Bootstrapping vs Monte Carlo — a key distinction

Monte Carlo (no bootstrap)

Wait until episode ends.
Use actual total return G_t as target.
Unbiased, high variance.
Can only learn after full episodes.

TD (bootstrap)

Update after every single step.
Use $r + \gamma \max Q$ as target.
Initially biased, low variance.
Learns online, step by step.

Physics analogy — Self-consistent field (Hartree-Fock): Compute the mean field from your current wavefunction ψ, update ψ using that field, recompute the field… iterate to convergence. Q-learning is the same self-consistency loop in policy space.

Step 7 — Why No P(s'|s,a) Is Needed

Model-based (requires P):

$Q^*(s,a) = R(s,a) + \gamma \Sigma_{s'} P(s'|s,a) \max_{a'} Q^*(s',a')$

Must loop over all s' and weight by P. Intractable without the model.

Model-free (Q-learning):

$Q \leftarrow Q + \alpha[\delta]$

The environment samples $s' \sim P(\cdot|s,a)$ automatically. Each step = one unbiased sample of $\Sigma P[\ldots]$.

Why this works: three observations

1. When you execute action a in state s, the environment draws s' from exactly $P(\cdot|s,a)$ — the distribution we need but never have to write down.
2. This single s' is an unbiased sample of the expectation $\Sigma P[\ldots]$. The update $Q \leftarrow Q + \alpha\delta$ is a noisy but unbiased gradient step.
3. Over many visits to (s,a), the noise averages out by the law of large numbers. Q converges to the unique fixed point Q^*.

Watkins' key insight (1989): An agent does not need a model of the environment to learn the optimal policy. The environment itself provides all necessary information through the sequence of transitions it generates.

Step 8 — Convergence: The Fine Print

Robbins-Monro conditions on the learning rate α_t

The stochastic approximation theorem (Robbins & Monro 1951) guarantees Q → Q* if:

Condition 1 — divergent sum:

$\Sigma_t \alpha_t = \infty$

Enough total learning to correct any initialisation error.

Condition 2 — square-summable:

$\Sigma_t \alpha_t^2 < \infty$

Noise in the gradient estimate eventually vanishes.

The schedule $\alpha_t = 1/t$ satisfies both. A fixed α (e.g. 0.1) satisfies only Condition 1 — it doesn't converge to the exact Q* but tracks slowly-changing environments, which is often more useful in practice.

The exploration requirement (equally important)

Every (s,a) pair must be visited infinitely often. Without this, Q-table entries for unvisited pairs are never updated and remain at their initial values, regardless of how wrong they are. This is why ε-greedy exploration is not optional — it is a convergence requirement.

Practical recipe: Fixed α = 0.1 with ε-greedy annealing (ε: 1.0 → 0.05 over training). Satisfies the exploration condition, converges approximately, and adapts if the environment changes.

Summary — The Full Logical Chain

Bellman Fixed Point
$Q^* = T[Q^*]$

→ *can't compute P*

Expectation Form
$Q^* = E[y]$

→ *one sample (s,a,r,s')*

One-Sample Estimate
$\delta \approx E[\delta]$

→ *treat y as fixed*

SGD Step
$Q \leftarrow Q + \alpha\delta$

→ *stochastic approx.*

Q-Learning Rule
Watkins 1989

The three approximations — each individually justified

1. **P → single sample:** Replace ΣP[…] with one observed s'. Unbiased estimator. Noise averages out over repeated visits to (s,a).
2. **y treated as fixed:** Target y is held constant during the gradient step (semi-gradient). Not true gradient descent, but convergence still guaranteed by contraction.
3. **SGD step size α:** Update by αδ instead of exact minimisation. Convergence requires Robbins-Monro conditions and sufficient exploration (ε-greedy).

What makes Q-learning remarkable: each of the three approximations is individually justified. Together they convert an intractable fixed-point problem over an unknown model into a single one-line update rule that runs on raw experience.